Laura Hietala

**Näyttötehtävä 3**

Tietojenkäsittelyn koulutus
TKMI23KV

6.5.2023

Ohjelmoinnin perustaidot
Janne Turunen

**Tehtäväraportti**

Tässä tehtäväraportissa esittelen tekstin ja kuvankaappausten keinoin, kuinka olen toteuttanut kurssilla annetut ohjelmointitehtävät. Käytetty ohjelmointikieli on kurssin vaatimusten mukaisesti TypeScript.

**Osatehtävä A**

Toteuta funktio **laskePintaAla()**, joka laskee (suorakulmaisen) tontin pinta-alan ja palauttaa laskutuloksen halutussa muodossa. Esittele funktio nuoli-funktiona.

Funktiossa on oltava kolme parametria: tontin pituus metreinä, tontin leveys metrinä ja laskutuloksen muodon määritys. Laskutulokselle voi valita kolme eri muotoa: neliömetrit, aarit ja hehtaarit.

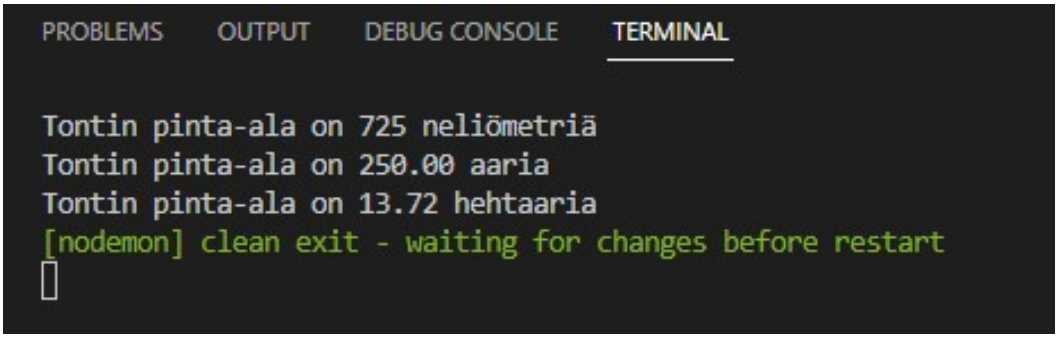
Testaa funktiosi toimivuutta erilaisilla testitulostuksilla kuten "Tontin pinta-ala on 3000 neliömetriä" tai "Tontin pinta-ala on 30 aaria". Raportoi myös testaus.

---

```
1  console.clear();
2
3  const laskePintaAla = (pituus: number, leveys: number, muoto: "neliömetrit" | "aarit" | "hehtaarit"): string => {
4      const pintaAla = pituus * leveys;
5      let tulos: string;
6
7      switch(muoto) {
8        case "neliömetrit":
9          tulos = `${pintaAla} neliömetriä`;
10         break;
11       case "aarit":
12         tulos = `${(pintaAla / 100).toFixed(2)} aaria`;
13         break;
14       case "hehtaarit":
15         tulos = `${(pintaAla / 10000).toFixed(2)} hehtaaria`;
16         break;
17     }
18
19     return `Tontin pinta-ala on ${tulos}`;
20   };
21
22
23   console.log(laskePintaAla(29, 25, "neliömetrit"));
24   console.log(laskePintaAla(100, 250, "aarit"));
25   console.log(laskePintaAla(490, 280, "hehtaarit"));
```

- Rivi 1: Tyhjennetään Terminal-ikkuna console.clear()-metodilla.
- Rivit 3-5: Määritellään funktio ”laskePintaAla” ja annetaan sille kolme parametria (pituus, leveys ja muoto). Funktio laskee pinta-alan kertomalla keskenään pituuden ja leveyden (metreinä) ja antaa tuloksen merkkijonona.
- Rivit 7-17: Käytetään muoto-parametrin määrittelyyn switch…case -ehtorakennetta, jolla ”muoto” voi saada kolme vaihtoehtoista pinta-alan määrittelyä (neliömetrit, aarit ja hehtaarit). Aarit saadaan jakamalla neliömetrit 100:lla (rivi 12), käytetään tässä .toFixed()-metodia pyöristämään tulosta kahteen desimaaliin. Hehtaarien kohdalla (rivi 15) tehdään muuten täsmälleen sama kuin aarien kanssa, mutta jaetaan neliömetrit 10 000:lla. Neliömetrien kanssa ei tarvita .toFixed()-metodia, sillä oletus on, että pituus ja leveys annetaan kokonaisina metreinä, jolloin neliömetrien pinta-alan laskutulos on aina kokonaisluku.
- Rivi 19: Pyydetään funktiota palauttamaan merkkijono, johon tulee template stringin avulla tontin pinta-ala toivotussa muodossa.
- Rivi 20: Funktio päättyy.
- Rivit 23-25: Testitulostusrivejä, joilla testataan funktion toimivuutta eri pituuksilla, leveyksillä ja pinta-alamuodoilla. Terminaaliin tulostuu funktion return-lause, kun funktiota kutsutaan console.log()-tulostuksessa ja funktion kolmen parametrin tuottama pinta-ala sekä muoto.

Näkymä terminaalissa tämän jälkeen:




**Osatehtävä B**

Toteuta luokka **Valuutta**, joka sisältää kaksi metodia:

1. **haeKurssi(),** jossa on määritelty parametri valuuttakoodille USD, GBP, SEK, NOK tai JPY. Metodi palauttaa halutun valuutan kurssin suhteessa euroon.
2. **muunna()** , jossa on määritelty kaksi parametria: euromäärä, joka halutaan muuntaa ja kohdevaluutan koodi: USD, GBP, SEK, NOK tai JPY. Metodi palauttaa annetun summan kohdevaluutassa.

Määritä vapaamuotoinen tietorakenne muuttujaan *valuuttakurssit*, joka sisältää valuuttakurssit annetulle viidelle valuutalle (USD, GBP, SEK, NOK ja JPY). Hae ajantasaiset kurssit verkosta.

Muodosta luokasta olio *valuuttalaskuri* käyttäen *valuuttakurssit*-muuttujaa argumenttina:

```
let valuuttalaskuri : Valuutta = new Valuutta(valuuttakurssit);
```

Testaa luokan/olion metodien toimivuutta erilaisilla argumenteilla ja raportoi myös tämä.

**HUOM!** Voit käyttää valuuttakurssien tyyppinä any-määritystä, jos kohtaat tässä kohdin ongelmia. Tyyppien välittäminen luokalle vaatii tyyppiparametrien käyttöä, jota emme käsittele tällä kurssilla. Toki saat selvittää asian itsenäisesti, mutta tämä ei ole pakollista pisteeseen.

---

```
1  console.clear();
2
3  interface Valuuttakurssit {
4      USD: number,
5      GBP: number,
6      SEK: number,
7      NOK: number,
8      JPY: number
9    }
10
11   class Valuutta {
12     valuuttakurssit: Valuuttakurssit;
13     EUR: number;
14
15     constructor(valuuttakurssit: Valuuttakurssit) {
16       this.valuuttakurssit = valuuttakurssit;
17       this.EUR = 1;
18     }
19
20     haeKurssi(valuutta: keyof Valuuttakurssit): number {
21       return this.valuuttakurssit[valuutta];
22     }
23
24     muunna(summa: number, kohdevaluutta: keyof Valuuttakurssit): number {
25       const kohdekurssi = this.haeKurssi(kohdevaluutta);
26       return summa * kohdekurssi;
27     }
28   }
```

```
29
30     let valuuttakurssit: Valuuttakurssit = {
31       USD: 1.10,
32       GBP: 0.88,
33       SEK: 11.30,
34       NOK: 11.47,
35       JPY: 146.89
36     };
37
38     let valuuttalaskuri: Valuutta = new Valuutta(valuuttakurssit);
39
40     console.log(valuuttalaskuri.muunna(100, "NOK"));
```

- Rivi 1: Tyhjennetään Terminal-ikkuna console.clear()-metodilla
- Rivit 3-9: Määritellään rajapinta Valuuttakurssit, joka määrittelee valuuttojen nimet ja niiden arvon (number).
- Rivit 11-13: Määritellään luokka ”Valuutta”, joka sisältää ominaisuuden valuuttakurssit (Valuuttakurssit-rajapinnan mukaisesti), ominaisuuden EUR (euro-valuutta, sillä valuuttalaskurissa muita valuuttoja muunnetaan suhteessa euroon) sekä konstruktori-metodin, jonka parametrina on Valuuttakurssit-rajapinta, joka alustaa valuuttakurssit-muuttujan. Lisäksi konstruktori antaa euron arvoksi 1.

- Rivit 20-22: Määritellään luokan Valuutta haeKurssi()-metodi, jonka parametrina on valuutta. Tämä parametri johdetaan Valuuttakurssit-rajapinnan avaimista eli valuutoista keyof-operaattoria käyttäen (en saanut Object.keysiä toimimaan tässä oikein, joten löysin internetin syövereistä keyof-operaattorin, joka onkin erittäin toimiva). Keyof-operaattori luo tyyppiunionin Valuuttakurssit-objektin avaimista. Metodi palauttaa valitun valuutan kurssin.
- Rivit 24-27: muunna-metodin parametreina on summa (alkuperäisen valuutan määrä) sekä kohdevaluutta (eli valuutta, johon tuo rahasumma muunnetaan). ottaa vastaan summan ja kohdevaluutan, muuntaa annetun summan euroiksi ja sitten kohdevaluutaksi kohdevaluutan valuuttakurssin perusteella. Metodi hakee haeKurssi() -metodin avulla kohdevaluutan kurssin taulukosta, jossa on lueteltu valuuttakurssit. Metodi palauttaa summan ja kohdekurssin tulon, eli alkuperäisen rahasumman muunnettuna kohdevaluutaksi.
- Rivi 28: Aaltosulku, joka sulkee class-lohkon.
- Rivit 30-36: Määritellään valuuttakurssit-objekti käyttämällä Valuuttakurssit-rajapintaa. Tässä objektissa määritellään kunkin valuutan kurssi suhteessa euroon (kurssit haettu 19.4.2023).
- Rivi 38: Uusi Valuutta-luokan ilmentymä, eli valuuttalaskuri-olio, joka käyttää valuuttakurssit-muuttujaa argumenttina.
- Rivi 40: Kutsutaan valuuttalaskurin muunna-metodia antamalla sille kaksi parametria (summa euroina ja kohdevaluutta), metodi muuntaa eurot kohdevaluutaksi. Koodi tulostaa tuloksen terminaaliin.

Testasin toimivuutta syöttämällä lukuja ja kohdevaluuttoja muuntimeen, ja koodi tuntui toimivan oikein. Se laski muunnokset täysin oikein.
Esim.:

```
console.log(valuuttalaskuri.muunna(100, "NOK"));
```
tulosti terminaaliin luvun 1147

```
console.log(valuuttalaskuri.muunna(70, "JPY"));
```
tulosti terminaaliin luvun 10282.3

```
console.log(valuuttalaskuri.muunna(1100, "USD"));
```
tulosti terminaaliin luvun 1210